

Universidad Nacional de Educación a Distancia
Departamento de Lenguajes y Sistemas Informáticos

FUNDAMENTOS DE INFORMÁTICA

Práctica Obligatoria - Curso 2023/2024

ÍNDICE GENERAL

INTRODUCCIÓN **2**

1.1. Descripción general de la práctica **2**

1.2. Diseño de la práctica: Diagrama de clases **3**

ETAPA 1 ..... **4**

2.1. ETAPA 1.1 - Orientación a Objetos: La clase Usuario **4**
2.1.1. Objetivos 4
2.1.2. Requisitos previos 4
2.1.3. Implementación de la clase Usuario 4

2.2. ETAPA 1.2 Ejecución de Aplicaciones: La clase principal **5**
2.2.1. Objetivos 5
2.2.2. Requisitos previos 5
2.2.3. Comprobación de la implementación de la clase principal 6

ETAPA 2 ..... **7**

3.1. ETAPA 2.1: Uso de la composición **7**
3.1.1. Objetivos 7
3.1.2. Requisitos previos 7
3.1.3. Creación de la clase Producto y modificación de la clase Usuario 7
3.1.4. Creación de la clase PlataformaCompraVenta (incluye parte Opcional) 9
3.1.5. Comprobación del funcionamiento del mecanismo de composición y de los métodos de listado de usuarios y productos. 9
3.1.6. Comprobación del funcionamiento de las funcionalidades más avanzadas (parte Opcional). 16

ETAPA 3 ..... **22**

4.1. ETAPA 3.1: Uso de la herencia y polimorfismo **22**
4.1.1. Objetivos 22
4.1.2. Requisitos previos 22
4.1.3. Añadiendo productos de diferentes tipos 22

FECHAS Y NORMAS DE ENTREGA **29**

5.1. Sesiones presenciales **29**

5.2. Normas de entrega **29**

5.3. Fechas de entrega **29**

5.3. Evaluación de la práctica **30**

5.4. Preguntas al equipo docente **30**

INTRODUCCIÓN

La práctica se realizará utilizando *Java* como lenguaje de Programación Orientado a Objetos. El primer concepto que se debe conocer es el de *objeto*, el ladrillo con el que se construye un programa bajo este paradigma de programación. Los objetos tienen vida cuando nuestro programa está en ejecución, y pueden relacionarse entre sí de diversas maneras, proporcionando al programador las funcionalidades necesarias para resolver una amplia gama de problemas.

La definición de un objeto viene dada por la *clase*, siendo esta la entidad que en realidad diseñaremos y programaremos en términos de sus *campos* (o *atributos*) y *métodos*. Estos campos y métodos nos permiten definir las características y funcionalidades de la clase. Una vez definida, a partir de la clase podremos crear objetos, que son los que en realidad llevarán a cabo la funcionalidad de nuestro programa en tiempo de ejecución.

Por tanto, podemos ver los objetos como los componentes que nos servirán para diseñar nuestro programa, y las clases como las definiciones de dichos objetos. Una misma clase puede dar lugar a muchos objetos diferentes, cada uno con características diferentes que vendrán dadas por los valores de sus campos.

1.1. Descripción general de la práctica

El propósito de esta práctica es que el alumno desarrolle un sistema informático que simule parte del funcionamiento de una plataforma online para la compraventa de productos de segunda mano, al estilo de Wallapop[1]. Este proyecto busca familiarizar al estudiante con la creación de aplicaciones informáticas, llevando a la práctica los principios de programación en Java.

En esta práctica se espera que los estudiantes implementen una versión muy simplificada de la plataforma, que solo tendrá como objeto mostrar un catálogo de productos en venta ofertados por un conjunto de usuarios dados de alta en el sistema.

Se considerarán únicamente dos categorías de productos, ropa y electrónica, y cada producto estará caracterizado por su precio, una descripción y, dependiendo de la categoría, el tamaño o sus dimensiones físicas. Los productos tendrán dos estados posibles: *disponible* o *reservado*. Por otro lado, los usuarios que ofertan los productos en esta plataforma serán identificados por su correo electrónico de contacto y su nombre, y deberán tener asociados los productos que ofertan para su venta.

Por tanto, el sistema no contempla ningún tipo de transacción de compra y venta de productos; se trata de un sistema que permitirá mostrar un catálogo de productos en venta asociados a usuarios. El correo electrónico del usuario que oferta un producto sería la forma de contacto posterior con la que se realizaría la compraventa.

Las funcionalidades básicas que se deberán implementar en esta plataforma serán las siguientes:

1. **Gestión de usuarios**. La plataforma ofrecerá la posibilidad de añadir y eliminar usuarios dentro del sistema. Los usuarios son aquellas personas que ponen a la venta algún producto dentro de alguna de las dos categorías consideradas.
2. **Gestión de productos**. Se deberá permitir que los usuarios agreguen y retiren productos para su venta.
3. **Visualización de productos**. Se podrán mostrar por pantalla los detalles de los diferentes productos en venta, así como del usuario que los oferta, de modo que si el producto se encuentra en estado *disponible* se pueda contactar con el usuario por medio de correo electrónico para realizar la compraventa; como se ha comentado, esta última parte quedaría fuera del alcance de esta práctica.
4. **Visualización de información de usuarios**. La plataforma permitirá mostrar por pantalla la información de cualquier usuario, incluyendo los productos que ofrece, con sus características y su estado.
5. **Búsqueda por usuario y presentación de productos**. Será posible buscar los productos ofertados por un usuario, mostrando aquellos que estén disponibles o reservados.
6. **Búsqueda por palabra clave**. Se permitirá la búsqueda de productos que contengan palabras clave específicas

[1] https://es.wallapop.com

en su descripción, y se podrán mostrar en pantalla.

Este ejercicio de programación se centrará únicamente en el *back-end*, es decir, en la parte del desarrollo encargada de la lógica necesaria para que el sistema funcione, omitiendo la interacción real con una interfaz de usuario y las comunicaciones de red; todo ello será obviado en esta práctica. De este modo, el objetivo es completar la programación de la lógica que subyace a la plataforma de comercio electrónico planteada, poniendo a prueba la capacidad de los estudiantes para estructurar y organizar el código de una manera que facilite el crecimiento iterativo y la adición de nuevas funcionalidades en futuras versiones.

Para la realización de esta práctica se seguirá un enfoque iterativo, de forma que el desarrollo se dividirá en etapas. En cada etapa se partirá del diseño y desarrollo de la etapa inmediatamente anterior, añadiéndole nuevas funcionalidades y llevando a la práctica los conceptos del temario que se van estudiando progresivamente.

Además, el enunciado de la práctica se acompaña de distintos fragmentos de código, que servirán para validar cada una de las etapas del enunciado, lo que permitirá comprobar si la solución que se está dando en cada momento es correcta o no. Ese código de comprobación estará encapsulado en distintas clases, una por cada etapa, planteando un conjunto de pruebas que permitirán validar la solución propuesta hasta ese momento. Las pruebas consistirán en llamadas a métodos de los objetos creados en cada etapa, por lo que **será muy importante respetar y seguir los identificadores que se proponen para los distintos elementos; deberán respetarse el nombre de las clases, atributos, métodos, tipos de datos, etc**.

RECOMENDACIÓN: la práctica se realiza a lo largo del cuatrimestre y forma parte del estudio de la asignatura. Es conveniente hacer una primera lectura completa de este documento para tener una visión global de lo que se pide, pudiéndose organizar así apropiadamente el estudio de la asignatura.

1.2. Diseño de la práctica: Diagrama de clases

Cuando afrontamos el desarrollo de una aplicación informática es indispensable y necesario realizar una fase previa donde se analicen los requisitos y se diseñen las entidades; en nuestro caso, las clases necesarias para llevar a cabo las funcionalidades requeridas y la relación entre las mismas. Para ello, antes de implementar cada etapa en las que se ha dividido la práctica, se deberá realizar una lectura completa de la etapa a abordar, realizando un diagrama donde se representen las clases necesarias con sus atributos y métodos, así como las relaciones entre las mismas, para posteriormente comenzar la fase de implementación. Así, se deberá realizar un diagrama de clases desde la primera etapa, que luego posteriormente se irá refinando y/o modificando en cada una de las siguientes etapas, siempre como requisito previo a comenzar a implementar el código de cada etapa.

El diagrama de clases obtenido en la última etapa debe ser entregado junto al código de la práctica.

ETAPA 1

2.1. ETAPA 1.1 - Orientación a Objetos: La clase Usuario

2.1.1. Objetivos

En esta primera etapa de la práctica se van a afianzar los conceptos básicos de la programación orientada a objetos: **clase**, **objeto**, **atributos** y **métodos** (constructores, métodos que ayudan a la encapsulación de la clase y métodos de visualización). Además, se hará uso de tipos enumerados para definir el conjunto de valores que puede tomar un atributo.

Veámoslo con un ejemplo que nos permita comenzar esta primera etapa de la práctica. Este año modelaremos una plataforma de compraventa online de productos de diferentes categorías, que podrán darse de alta o baja por parte del usuario que los oferta, podrá visualizarse su descripción, etc. Por tanto, una entidad básica en este dominio es el usuario. Para ello, vamos a empezar diseñando e implementando una clase denominada `Usuario`, que nos permitirá encapsular la información asociada a un usuario que pone en venta productos dentro de nuestra plataforma. Así, cada usuario dado de alta en el sistema se modelará mediante un objeto de clase, con características comunes (atributos o campos), y valores podrán ser diferentes, representando así cada usuario dentro de la plataforma.

2.1.2. Requisitos previos

Para la realización de esta primera etapa se requiere haber estudiado los temas 4, 5 y 6 del temario detallado de la asignatura, correspondientes a los capítulos 1, 2 y 3, así como el apartado 6.13.1 y los apéndices A, B, C, D y F del libro de referencia de la Unidad Didáctica II.

2.1.3. Implementación de la clase Usuario

En esta primera etapa realizaremos el modelado inicial de alguno de los componentes principales del sistema. Uno de estos elementos es el usuario, que será modelado como una clase a partir de la cual se podrán crear instancias u objetos de la misma. En esta fase del desarrollo de la práctica, cada usuario se describe por la siguiente información, que modelaremos como atributos de la clase `Usuario`:

- `nombre`, que indicará el nombre del usuario que pone un producto en venta dentro de la plataforma.
- `email`, el correo electrónico por medio del cual se realizará el contacto en caso de que se quisiera realizar la compra de un producto ofertado por el usuario, aunque como ya se ha avanzado esta parte de la venta queda fuera del alcance de la práctica.

Además, la clase `Usuario` deberá permitir consultar y modificar estos campos a través de una serie de métodos creados a tal efecto. Por cada atributo debe haber dos métodos: uno para consultar su valor y otro para modificarlo. Por ejemplo, para el campo `nombre` tendremos los métodos `getNombre()` y `setNombre()` para consultar su valor y modificarlo, respectivamente. Finalmente, la clase `Usuario` dispondrá también de un método `print()` que imprima en pantalla toda su información.

Presentamos a continuación en la Figura 2.1 un ejemplo de formato que debería tener la salida en pantalla de este método.

```
Nombre: <nombre>
Email: <email>
```

Figura 2.1: Ejemplo de formato de salida por pantalla mostrando la información de un producto en venta.

RECOMENDACIÓN: practicar con *BlueJ* creando diferentes objetos de la clase *Producto*, modificando el valor de sus atributos, mostrándolos por pantalla, etc.

2.2. ETAPA 1.2 Ejecución de Aplicaciones: La clase principal

2.2.1. Objetivos

Es importante comprender que Java y *BlueJ* son cosas diferentes. Java es un lenguaje de programación, mientras que *BlueJ* es un entorno de programación que nos permite programar en Java. *BlueJ* está pensado para el aprendizaje de Java y proporciona distintas herramientas que permiten inspeccionar las partes de un programa, tanto en la parte de diseño (cuando estamos escribiendo el código) como en la de ejecución (cuando el código se ejecuta, se crean los objetos en memoria, etc.)

Una de las herramientas de *BlueJ* que pueden resultar más útiles es el banco de objetos u *object bench* (ver capítulo 1 del libro), que junto con otras herramientas como el *inspector de objetos*, resulta muy interesante para "jugar" con los objetos en tiempo de ejecución de forma interactiva. Podemos ver un ejemplo de estas herramientas en la Figura 2.2, donde se muestran varios objetos de una clase `Pelicula` en el banco de objetos; en el ejemplo, uno de ellos está siendo inspeccionado, lo que nos permite visualizar los valores de sus atributos. La clase `Pelicula` no está incluida en la práctica; es solo un ejemplo para mostrar el funcionamiento de *BlueJ*.

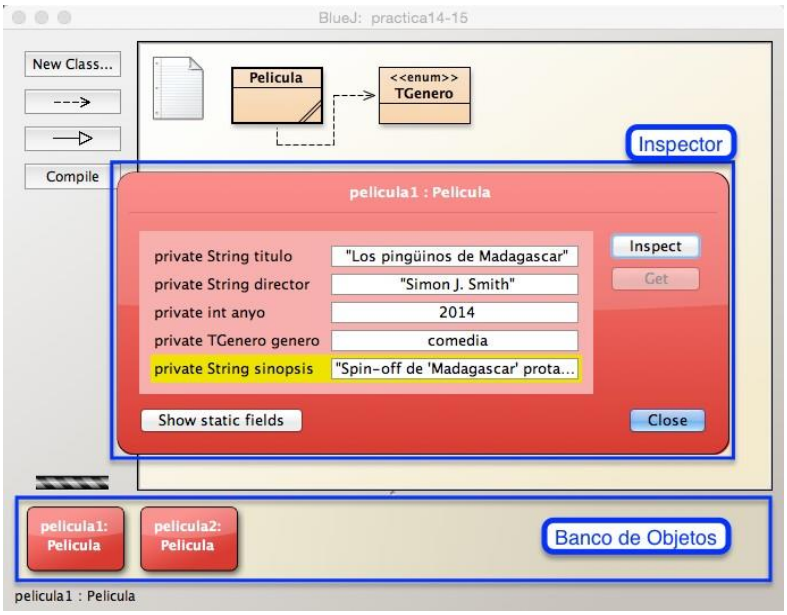


Figura 2.2: Interfaz de *BlueJ* donde se pueden ver dos películas creadas en el banco de objetos.

BlueJ nos permite ejecutar nuestras aplicaciones mediante la invocación de los métodos de sus objetos, pero no es esta la única manera de hacerlo. Java, más allá de *BlueJ*, nos permite ejecutar los programas que creamos de forma independiente. A estas alturas sabemos que cada clase tiene unos métodos y campos propios de los objetos creados a partir de dicha clase, que para poderlos usar necesitamos crear primero un objeto. Por ejemplo, la clase `Pelicula` que hemos creado tiene un método para consultar su `titulo`, que sólo podría ser invocado una vez tengamos creado algún objeto de esta clase.

RECOMENDACIÓN: Al final de este apartado se debería entender la diferencia entre ejecutar un programa en *BlueJ* y hacerlo de forma independiente; esto es, ejecutarlo desde la línea de comandos (por ejemplo, desde la consola o el símbolo de sistema en *Windows*/*Linux*/*MacOS*).

2.2.2. Requisitos previos

Además de los expuestos en la sección 2.2.1, deben consultarse capítulos 6.15 y 6.16 del libro de texto, sobre métodos de clase. Además, en el apéndice E puede encontrar más información sobre el método `main()`, en concreto en el apartado E1. Puede consultar también el capítulo 3.10 "Objetos que crean objetos" si necesita más información acerca de la creación de objetos desde el código.

2.2.3. Comprobación de la implementación de la clase principal

Para ejecutar nuestro programa de manera independiente de *BlueJ* y crear el flujo principal del sistema, crearemos una clase llamada `LanzadorEtapa1` e implementaremos su método `main()`. Esta clase contendrá el código en el que se crean las instancias de objetos necesarias y las llamadas para poder comprobar el funcionamiento del sistema.

El ejercicio a realizar en este punto consiste en crear dicha clase `LanzadorEtapa1`, que contenga un método `main()` e implementar el siguiente código[2]:

```
public class LanzadorEtapa1 {
    public static void main (String [ ] args) {

Usuario user1 = new Usuario("Mariano Pérez Oso","marian0@hismail.com");
Usuario user2 = new Usuario("Javier Carro Ñero","javicarronyero@othermail.com");
Usuario user3 = new Usuario("Antonio N. Migo","antonomigo@mymail.com");
user1.print();
user2.print();
user3.print();
}}
```

Este código debería crear tres instancias de la clase `Usuario`. Posteriormente, el código imprime por pantalla la información de todos los usuarios creados.

Al ejecutar el `main()` de este `LanzadorEtapa1`, el sistema deberá imprimir por pantalla la siguiente información:

```
Nombre: Mariano Pérez Oso
Email: marian0@hismail.com

Nombre: Javier Carro Ñero
Email: javicarronyero@othermail.com

Nombre: Antonio N. Migo
Email: antonomigo@mymail.com
```

Se recomienda probar con más objetos y valores de sus atributos.

Como ejercicio, debe ejecutarse esta clase `LanzadorEtapa1`, comprobando que compila y ejecuta correctamente, visualizándose los datos de los usuarios. A continuación, modifique el método `main()` para crear nuevos objetos de la clase `Usuario`, asignando los valores que desee para sus atributos y visualícelos de manera análoga. Ejecute el programa desde *BlueJ* y desde la línea de comandos (consola o símbolo de sistema, según el sistema operativo que utilice).

NOTA IMPORTANTE: La presente práctica debe poderse ejecutar independientemente de *BlueJ* de cara a su entrega y corrección.

[2] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase `LanzadorEtapa1`.

ETAPA 2

3.1. ETAPA 2.1: Uso de la composición

3.1.1. Objetivos

Una manera muy usual de que los objetos se relacionen en la programación orientada a objetos es mediante el mecanismo de **composición**. Esta relación se produce cuando un objeto contiene a otros objetos para realizar alguna tarea con ellos. En Java tenemos distintas formas de aplicar este mecanismo; por un lado, podemos tener un objeto como atributo de una clase, pero también podemos emplear estructuras de datos que pueden almacenar conjuntos de objetos, permitiéndonos realizar otro tipo de relaciones de composición. Algunos ejemplos de estas estructuras son: las listas, los conjuntos o los mapas.

En esta segunda etapa de la práctica haremos uso de la composición de la siguiente forma:

- En primer lugar, modificando la definición de la clase `Usuario` para incluir un atributo con la lista de productos que quiere poner en venta dentro de la plataforma.
- En segundo lugar, la composición se aplicará para integrar una nueva clase que habrá que crear, la clase `PlataformaCompraVenta`, y que será la encargada de gestionar las altas y bajas de cada uno de los usuarios dentro en la plataforma, así como de asignar productos a usuarios. Permitirá también acceder a información de productos a través de los usuarios que los ofertan.

En resumen, deberá crearse una nueva clase `PlataformaCompraVenta` que tendrá como atributo una lista de objetos de la clase `Usuario`. También será necesario modificar la clase `Usuario` creada en la etapa anterior para incluir como campo una lista de objetos de una nueva clase `Producto` que también deberemos crear.

En esta etapa se aplicarán los principios de encapsulación de la programación orientada a objetos, según se describe en el libro base de la asignatura. Esto implica que los atributos de las clases deben estar definidos como **privados**, siendo los métodos de la propia clase los únicos que tienen acceso a dichos atributos.

Finalmente, es una buena práctica de programación documentar todo el código que se va escribiendo, no solo con el objetivo de que terceras personas puedan comprender nuestro programa, sino también para que nosotros mismos podamos entender nuestro código de cara a futuras extensiones, mejoras o modificaciones. Para la generación de documentación, Java nos ofrece la herramienta **Javadoc**, que por medio de determinado tipo de comentarios añadidos en nuestro código fuente, nos permite la generación posterior de toda la documentación de las distintas clases.

3.1.2. Requisitos previos

Para la realización de esta etapa se deben estudiar los temas 7, 8 y 9 del temario detallado de la asignatura. Encontrará información más detallada sobre el mecanismo de composición y la generación de documentación en el libro de texto de la asignatura en lo capítulos 4, 5 y 6. La lectura de las secciones de 8.3 a 8.6 (ambas inclusive) sobre buenas prácticas en el diseño de aplicaciones es también recomendable.

RECOMENDACIÓN: Lea primero los requisitos completos antes de tomar decisiones acerca de cómo desarrollar la solución.

3.1.3. Creación de la clase Producto y modificación de la clase Usuario

Nuestra aplicación permitirá gestionar la oferta de productos de segunda mano que modelaremos mediante una clase `Producto`. Pero hasta ahora, la clase `Usuario` no tenía ningún atributo para guardar los productos que ofertaba un usuario, de modo que también habrá que modificar esta clase para incluirlo. Ahora queremos que `Usuario` tenga como atributo una lista de objetos de una clase `Producto`.

Por tanto, en primer lugar, deberemos crear una clase `Producto` con los siguientes atributos:

- `id`, que permitirá identificar unívocamente un producto dentro del sistema.
- `titulo`, que indicará el texto principal que se pondrá en el anuncio.
- `precio`, que establecerá el precio de venta del producto de segunda mano.
- `descripcion`, que contendrá el texto que describe las características del producto en venta.
- `estado`, que mostrará el estado actual del producto en venta. Para ello usaremos un tipo enumerado `T_Estado` que habrá que crear con los valores posibles: `DISPONIBLE` o `RESERVADO`.

Aparte de los campos anteriores, habrá que implementar métodos que permitan consultar y modificar la información de todos ellos. También deberá poder mostrarse por pantalla la información completa de un producto por medio de un método `print()`. El formato de salida por pantalla será el mostrado en la Figura 2.3.

```
ID: <id>
Producto en venta: <titulo>
Precio: <precio>
Descripción:<descripcion>
Estado: <DISPONIBLE o RESERVADO>
```

Figura 2.3: Ejemplo de formato de salida por pantalla mostrando la información de un producto en venta.

Una vez creada la clase `Producto`, se deberá incluir un atributo `productos` en la clase `Usuario` para almacenar los productos que oferte un usuario. Este atributo deberá ser una lista de objetos de la clase `Producto`. Se deberá permitir añadir y eliminar productos a esta lista, de modo que en el momento en el que se incluya un producto en la lista se fije por defecto su estado como *disponible*.

El método `print()` de la clase `Usuario` deberá ser modificado también para mantener un formato de salida similar al mostrado en la Figura 2.1. La Figura 2.4 muestra un ejemplo del formato que deberá tener ahora la salida por pantalla del método en esta etapa 2, incluyendo también las descripciones de todos los productos que tiene en venta del usuario.

```
Nombre y apellidos: <nombre>
Email: <email>

ID: <id>
Producto en venta: <titulo>
Precio: <precio>
Descripción:<descripcion>
Estado: <DISPONIBLE o RESERVADO>

ID: <id>
Producto en venta: <titulo>
Precio: <precio>
Descripción:<descripcion>
Estado: <DISPONIBLE o RESERVADO>

ID: <id>
Producto en venta: <titulo>
Precio: <precio>
Descripción:<descripcion>
Estado: <DISPONIBLE o RESERVADO>
```

Figura 2.4: Ejemplo de formato de salida por pantalla mostrando la información de un usuario en la etapa 2. Nótese que en este caso el usuario tendría 3 productos en venta.

3.1.4. Creación de la clase PlataformaCompraVenta (incluye parte Opcional)

Luego se creará una clase que permita la gestión del sistema, la gestión de usuarios y sus productos en venta. Nuestra plataforma de compraventa se gestionará por medio de una clase `PlataformaCompraVenta`, que deberá contar entonces con un atributo `usuarios` para guardar la lista de usuarios del sistema. Incluiremos también otro atributo `nombre` para denominar nuestra plataforma de compraventa.

El comportamiento de esta clase que permite la gestión del sistema debe cubrir las siguientes funcionalidades:

1. Se podrá dar de alta un nuevo usuario en el sistema con toda la información correspondiente al mismo, es decir, se deberá permitir añadir un objeto de la clase `Usuario` al atributo `usuarios`.
2. Se permitirá eliminar usuarios del sistema a partir de su `email`.
3. Se podrá asociar un producto a un usuario dado su email. De modo análogo, se deberá permitir eliminar un producto de lista de productos de un usuario, dados el `id` del producto y el `email` del usuario.
4. Se podrá listar por pantalla el conjunto de productos que se encuentran en venta en la plataforma, así como el de usuarios dados de alta.

(Parte Opcional)

> *Esta parte de la práctica correspondiente a las funcionalidades 5, 6 y 7 es opcional. La correcta solución de la misma permitirá al estudiante optar a la máxima calificación de la práctica (Sobresaliente), no permitiendo compensar deficiencias en otras partes obligatorias del enunciado, siendo un notable la máxima calificación a la que se puede optar sin realizar esta parte.*

5. Se permitirá también imprimir el listado según del estado de los productos, es decir, el conjunto de todos los productos *disponibles* o *reservados* dentro del sistema.
6. Se podrá listar por pantalla el conjunto de productos que ofrece un determinado usuario, con un determinado estado y a partir de su correo electrónico.
7. Por último, se permitirá buscar productos dentro del sistema a partir de una palabra clave, mostrando por pantalla la información de cada uno de ellos.

En la creación de esta clase que permite la gestión del sistema es crucial tener en cuenta el concepto de encapsulación. La clase `PlataformaCompraVenta` se encarga de mantener la lista de usuarios dados de alta dentro del sistema. Adelantamos que en la etapa posterior querremos considerar diferentes categorías de productos con características distintas; en términos de programación orientada a objetos esto implica que la información que vayamos a imprimir por pantalla de cada producto será diferente en un caso u otro.

Llegados a este punto del desarrollo de la práctica, deben documentarse todas las clases creadas hasta el momento. En el capítulo 6, concretamente en la sección 6.11, se describen los pasos a seguir para documentar las clases correctamente.

3.1.5. Comprobación del funcionamiento del mecanismo de composición y de los métodos de listado de usuarios y productos.

Vamos a crear ahora una nueva clase llamada `LanzadorEtapa2_1`, que contenga un método `main()` e implementar en este método el siguiente código[3]:

[3] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase `LanzadorEtapa2`.

```
public class LanzadorEtapa2_1 {
    public static void main(String[] args) {

        // Creamos la plataforma de compraventa
        System.out.println("-----------------------------------");
        System.out.println("Creamos la plataforma de compraventa");
        System.out.println("-----------------------------------");
        PlataformaCompraVenta gestor = new PlataformaCompraVenta("Wallarock");

        // Creamos usuarios
        System.out.println("----------------");
        System.out.println("Creamos usuarios");
        System.out.println("----------------");
        Usuario user1 = new Usuario("Mariano Pérez Oso","marian0@hismail.com");
        Usuario user2 = new Usuario("Javier Carro Ñero","javicarronyero@othermail.com");
        Usuario user3 = new Usuario("Antoine Val Ondeoro ","antoine@hismail.com");
        Usuario user4 = new Usuario("Martín Antonio García","mantoniog@othermail.com");
        // añadimos usuarios a la plataforma
        System.out.println("----------------");
        System.out.println("Añadimos usuarios");
        System.out.println("----------------");
        gestor.addUser(user1);
        gestor.addUser(user2);
        gestor.addUser(user3);
        gestor.addUser(user4);

        // Creamos productos
        System.out.println("----------------");
        System.out.println("Creamos productos");
        System.out.println("----------------");
        Producto producto1 = new Producto(1,"Cámara digital Samsung",85,"Cámara digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.",T_Estado.DISPONIBLE);
        Producto producto2 = new Producto(2,"Mesa de mezclas analogica Behringer Xenyr 1202",110,"Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta.",T_Estado.DISPONIBLE);
        Producto producto3 = new Producto(3,"Cámara digital Panasonic",65,"cámara Panasonic mini c 525AF analógica 35mm 1:3.5 + funda original.",T_Estado.DISPONIBLE);
        Producto producto4 = new Producto(4,"Mini video cámara digital deportiva",20,"video cámara unotec gartner",T_Estado.DISPONIBLE);
        Producto producto5 = new Producto(5,"Cámaras foto digital",40,"No hago envios(batería no funciona). El resto de la cámara está como nueva",T_Estado.DISPONIBLE);
```

```
Producto producto6 = new Producto(6,"Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S",649.49,"Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).",T_Estado.DISPONIBLE);

// Añadimos los productos a los usuarios
System.out.println("-----------------------------------");
System.out.println("Añadimos los productos a los usuarios");
System.out.println("-----------------------------------");
gestor.addProduct2User(producto1,"marian0@hismail.com");
gestor.addProduct2User(producto2,"javicarronyero@othermail.com");
gestor.addProduct2User(producto3,"antoine@hismail.com");
gestor.addProduct2User(producto4,"antoine@hismail.com");
gestor.addProduct2User(producto5,"antoine@hismail.com");
gestor.addProduct2User(producto6,"mantoniog@othermail.com");

// Imprimimos la info de un producto
System.out.println("--------------------------------");
System.out.println("Imprimimos la info de un producto");
System.out.println("--------------------------------");
producto2.print();

// Imprimimos la info de un usuario
System.out.println("--------------------------------");
System.out.println("Imprimimos la info de un usuario");
System.out.println("--------------------------------");
user3.print();

// Eliminamos un usuario a partir de su email
System.out.println("----------------------------------------");
System.out.println("Eliminamos un usuario a partir de su email");
System.out.println("----------------------------------------");
gestor.removeUser("antoine@hismail.com");

// Imprimimos la lista de usuarios dados de alta
System.out.println("------------------------------");
System.out.println("Imprimimos la lista de usuarios");
System.out.println("------------------------------");
gestor.printAllUsers();


// Imprimimos el conjunto total de productos que se encuentran en venta en la plataforma
System.out.println("---------------------------------------");
System.out.println("Imprimimos el total de productos en venta");
System.out.println("---------------------------------------");
```

```
                gestor.printAllProducts();
        }

    }
```

Al ejecutar la clase anterior:

1. Se crea un objeto de la clase `PlataformaCompraVenta` y se imprime lo siguiente por pantalla:

```
-----------------------------------
Creamos la plataforma de compraventa
-----------------------------------
Creada plataforma de compraventa Wallarock
```

2. Se crean diferentes usuarios y se dan de alta en el sistema, mostrándose por pantalla lo siguiente:

```
----------------
Creamos usuarios
----------------
Creado usuario marian0@hismail.com
Creado usuario javicarronyero@othermail.com
Creado usuario antoine@hismail.com
Creado usuario mantoniog@othermail.com
----------------
Añadimos usuarios
----------------
El usuario con email marian0@hismail.com ha sido añadido
El usuario con email javicarronyero@othermail.com ha sido añadido
El usuario con email antoine@hismail.com ha sido añadido
El usuario con email mantoniog@othermail.com ha sido añadido
```

3. Se crean productos, y a continuación se asocian a diferentes usuarios, mostrándose:

```
----------------
Creamos productos
----------------
Creado producto con id 1
Creado producto con id 2
Creado producto con id 3
Creado producto con id 4
Creado producto con id 5
Creado producto con id 6
--------------------------------------
Añadimos los productos a los usuarios
```

El estado del producto con id 1 ha sido modificado a estado DISPONIBLE

El producto con id 1 se ha añadido al usuario marian0@hismail.com

El estado del producto con id 2 ha sido modificado a estado DISPONIBLE

El producto con id 2 se ha añadido al usuario javicarronyero@othermail.com

El estado del producto con id 3 ha sido modificado a estado DISPONIBLE

El producto con id 3 se ha añadido al usuario antoine@hismail.com

El estado del producto con id 4 ha sido modificado a estado DISPONIBLE

El producto con id 4 se ha añadido al usuario antoine@hismail.com

El estado del producto con id 5 ha sido modificado a estado DISPONIBLE

El producto con id 5 se ha añadido al usuario antoine@hismail.com

El estado del producto con id 6 ha sido modificado a estado DISPONIBLE

El producto con id 6 se ha añadido al usuario mantoniog@othermail.com

4. A continuación, se imprime por pantalla la información de un producto, y de un usuario y sus productos:

Imprimimos la info de un producto

ID: 2

Titulo: Mesa de mezclas analogica Behringer Xenyr 1202

Precio: 110.0

Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal

Estado: DISPONIBLE

Imprimimos la info de un usuario

Nombre: Antoine Val Ondeoro

Email: antoine@hismail.com

ID: 3

Titulo: Cámara digital Panasonic

Precio: 65.0

Descripcion: cámara Panasonic mini c 525AF analógica 35mm 1:3.5 + funda original.

Estado: DISPONIBLE

ID: 4
Titulo: Mini video cámara digital deportiva
Precio: 20.0
Descripcion: video cámara unotec gartner
Estado: DISPONIBLE

ID: 5
Titulo: Cámaras foto digital
Precio: 40.0
Descripcion: No hago envios(batería no funciona). El resto de la cámara está como nueva
Estado: DISPONIBLE

5. Se elimina un usuario del sistema y se imprime la lista de usuarios, junto con su información, que quedan dados de alta en la plataforma:

Eliminamos un usuario a partir de su email

El usuario con email antoine@hismail.com ha sido eliminado

Imprimimos la lista de usuarios

Nombre: Mariano Pérez Oso
Email: marian0@hismail.com

ID: 1
Titulo: Cámara digital Samsung
Precio: 85.0
Descripcion: Cámara digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.
Estado: DISPONIBLE

Nombre: Javier Carro Ñero
Email: javicarronyero@othermail.com

ID: 2
Titulo: Mesa de mezclas analogica Behringer Xenyr 1202
Precio: 110.0

Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal
Estado: DISPONIBLE

Nombre: Martín Antonio García
Email: mantoniog@othermail.com

ID: 6
Titulo: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S
Precio: 649.49
Descripcion: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).
Estado: DISPONIBLE

6. Por último, se imprime el total de productos que quedan en la plataforma:

Imprimimos el total de productos en venta

ID: 1
Titulo: Cámara digital Samsung
Precio: 85.0
Descripcion: Cámara digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.
Estado: DISPONIBLE

ID: 2
Titulo: Mesa de mezclas analogica Behringer Xenyr 1202
Precio: 110.0
Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal
Estado: DISPONIBLE

ID: 6

Titulo: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S

Precio: 649.49

Descripcion: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS.

(desgaste).

Estado: DISPONIBLE

Es importante probar el sistema con otros datos para comprobar su correcto funcionamiento.

3.1.6. Comprobación del funcionamiento de las funcionalidades más avanzadas (parte Opcional).

A continuación, vamos a probar el comportamiento del resto de funcionalidades que debe contemplar la clase `PlataformaCompraVenta`. Para ello vamos a crear una nueva clase `LanzadorEtapa2_2` que nos permita probar los métodos definidos para mostrar:

1. El conjunto de productos que se venden en la plataforma y el listado según su estado, los productos *disponibles* o *reservados* dentro del sistema.
2. El conjunto de productos que ofrece para su venta un usuario, con un determinado estado y a partir de su correo electrónico.
3. Los productos en venta en cuya descripción aparezca una determinada palabra clave.

El código siguiente, correspondiente a la clase `LanzadorEtapa2_2`, se encuentra también disponible en el curso virtual junto con el enunciado de la práctica. Como puede verse, incluye las instrucciones contenidas en el anterior lanzador, cuya salida ahora ya no será comentada.

```
public class LanzadorEtapa2 {

    public static void main(String[] args) {

        // Creamos la plataforma de compraventa
        System.out.println("-----------------------------------");
        System.out.println("Creamos la plataforma de compraventa");
        System.out.println("-----------------------------------");
        PlataformaCompraVenta gestor = new PlataformaCompraVenta("Wallarock");

        // Creamos usuarios
        System.out.println("----------------");
        System.out.println("Creamos usuarios");
        System.out.println("----------------");
        Usuario user1 = new Usuario("Mariano Pérez Oso","marian0@hismail.com");
        Usuario user2 = new Usuario("Javier Carro Ñero","javicarronyero@othermail.com");
        Usuario user3 = new Usuario("Antoine Val Ondeoro ","antoine@hismail.com");
        Usuario user4 = new Usuario("Martín Antonio García","mantoniog@othermail.com");
```

```
        // añadimos usuarios a la plataforma
        System.out.println("-----------------");
        System.out.println("Añadimos usuarios");
        System.out.println("-----------------");
        gestor.addUser(user1);
        gestor.addUser(user2);
        gestor.addUser(user3);
        gestor.addUser(user4);

        // Creamos productos
        System.out.println("-----------------");
        System.out.println("Creamos productos");
        System.out.println("-----------------");
        Producto producto1 = new Producto(1,"Cámara digital Samsung",85,"Cámara digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.",T_Estado.DISPONIBLE);
        Producto producto2 = new Producto(2,"Mesa de mezclas analogica Behringer Xenyr 1202",110,"Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \\\"británicos\\\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal",T_Estado.DISPONIBLE);
        Producto producto3 = new Producto(3,"Cámara digital Panasonic",65,"cámara Panasonic mini c 525AF analógica 35mm 1:3.5 + funda original.",T_Estado.DISPONIBLE);
        Producto producto4 = new Producto(4,"Mini video cámara digital deportiva",20,"video cámara unotec gartner",T_Estado.DISPONIBLE);
        Producto producto5 = new Producto(5,"Cámaras foto digital",40,"No hago envios(batería no funciona). El resto de la cámara está como nueva",T_Estado.DISPONIBLE);
        Producto producto6 = new Producto(6,"Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S",649.49,"Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).",T_Estado.DISPONIBLE);

        // Añadimos los productos a los usuarios
        System.out.println("-----------------------------------");
        System.out.println("Añadimos los productos a los usuarios");
        System.out.println("-----------------------------------");
        gestor.addProduct2User(producto1,user1);
        gestor.addProduct2User(producto2,user2);
        gestor.addProduct2User(producto3,user3);
        gestor.addProduct2User(producto4,user3);
        gestor.addProduct2User(producto5,user3);
        gestor.addProduct2User(producto6,user4);
```

```
// Imprimimos la info de un producto
System.out.println("--------------------------------");
System.out.println("Imprimimos la info de un producto");
System.out.println("--------------------------------");
producto2.print();

// Imprimimos la info de un usuario
System.out.println("--------------------------------");
System.out.println("Imprimimos la info de un usuario");
System.out.println("--------------------------------");
user3.print();

// Eliminamos un usuario a partir de su email
System.out.println("----------------------------------------");
System.out.println("Eliminamos un usuario a partir de su email");
System.out.println("----------------------------------------");
gestor.removeUser("antoine@hismail.com");

// Imprimimos la lista de usuarios dados de alta
System.out.println("-------------------------------");
System.out.println("Imprimimos la lista de usuarios");
System.out.println("-------------------------------");
gestor.printAllUsers();

// Imprimimos el conjunto total de productos que se encuentran en venta en la plataforma
System.out.println("----------------------------------------");
System.out.println("Imprimimos el total de productos en venta");
System.out.println("----------------------------------------");
gestor.printAllProducts();

// Eliminamos un producto a un usuario
System.out.println("---------------------------------");
System.out.println("Eliminamos un producto a un usuario");
System.out.println("---------------------------------");
gestor.removeProduct2User(1,user1.getEmail());
gestor.printProductsByUser(user1.getEmail());

System.out.println("-----------------------------");
System.out.println("Imprimimos todos los productos");
System.out.println("-----------------------------");
gestor.printAllProducts();
```

```
        // **************
        // PARTE OPCIONAL
        // **************

        producto2.setEstado(T_Estado.RESERVADO);

        // Imprimimos el conjunto total de productos que se encuentran en venta en la plataforma por estado
        System.out.println("----------------------------------------");
        System.out.println("Imprimimos los productos disponibles");
        System.out.println("----------------------------------------");

        gestor.printProductsByState(T_Estado.DISPONIBLE);


        System.out.println("-----------------------------------");
        System.out.println("Imprimimos los productos reservados");
        System.out.println("-----------------------------------");
        gestor.printProductsByState(T_Estado.RESERVADO);


        // Imprimimos el conjunto de productos que ofrece un determinado usuario
        System.out.println("------------------------------------------------");
        System.out.println("Imprimimos los productos disponibles de un usuario");
        System.out.println("------------------------------------------------");
        gestor.printProductsByUser(user4.getEmail(),T_Estado.DISPONIBLE);

        // Imprimimos los productos con una palabra clave
        System.out.println("------------------------------------------------------------");
        System.out.println("Imprimimos los productos que contienen la palabra \"nueva\" ");
        System.out.println("------------------------------------------------------------");
        gestor.searchProductsByKeyword("nueva");


    }

}
```

Al ejecutar la clase (y continuando a partir de la última instrucción contenida en el `LanzadorEtapa2_1`):

1. Se elimina un producto de la lista de un usuario y se imprime el conjunto total de productos dentro del sistema como comprobación:

```
---------------------------------
Eliminamos un producto a un usuario
```

El producto con id 1.0 se ha eliminado de la lista del usuario marian0@hismail.com

---

Imprimimos todos los productos

ID: 2

Titulo: Mesa de mezclas analogica Behringer Xenyr 1202

Precio: 110.0

Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal

Estado: RESERVADO

ID: 6

Titulo: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S

Precio: 649.49

Descripcion: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).

Estado: DISPONIBLE

2. Se modifica el estado de un producto, imprimiéndose el siguiente mensaje:

   El estado del producto con id 2 ha sido modificado a estado RESERVADO

3. Se imprimen por pantalla los productos incluidos en el sistema según su estado.

Imprimimos los productos disponibles

ID: 6

Titulo: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S

Precio: 649.49

Descripcion: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).

Estado: DISPONIBLE

Imprimimos los productos reservados

ID: 2
Titulo: Mesa de mezclas analogica Behringer Xenyr 1202
Precio: 110.0
Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal
Estado: RESERVADO

4. Se imprime el conjunto de productos disponibles de un usuario:

Imprimimos los productos disponibles de un usuario

ID: 6
Titulo: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom S
Precio: 649.49
Descripcion: Cámara digital EVIL Sony Alpha 6400 Negra + objetivo zoom Sony 16-50mm Incluye carcasa 2 BATERIAS. (desgaste).
Estado: DISPONIBLE

5. Se imprimen por pantalla los productos en cuya descripción aparce una determinada *keyword*.

Imprimimos los productos que contienen la palabra "nueva"

ID: 2
Titulo: Mesa de mezclas analogica Behringer Xenyr 1202
Precio: 110.0
Descripcion: Como nueva, con su caja y tranformador original. Behringer Xenyx 1202 Mesa de mezclas de 12 entradas con 2 buses, preamplificadores de micrófono Xenyx y ecualizadores británicos. Mesa de mezclas analógica de ultrabajo ruido con excelente margen de sobrecarga. 4 preamplificadores de micrófono XENYX comparables con preamplificadores de gama alta. Ecualizadores \"británicos\" neoclásicos de 3 bandas para un sonido cálido y musical. 1 envío post-fader para efectos externos en cada canal
Estado: RESERVADO

Es importante probar el código con otros datos, modificando el valor de las diferentes variables usadas para chequear el funcionamiento de los diferentes métodos y comprobar así el correcto funcionamiento del sistema.

ETAPA 3

4.1. ETAPA 3.1: Uso de la herencia y polimorfismo

4.1.1. Objetivos

En esta parte de la práctica vamos a extender el sistema para que nos permita gestionar productos de dos categorías diferentes: *ropa* y *electrónica*. Cada una de ellas incluirá nuevos atributos y variaciones en sus métodos. Para este tipo de situaciones, los lenguajes basados en el paradigma de la programación orientada a objetos, como Java, disponen de un mecanismo llamado *Herencia*. La herencia permite construir una nueva clase que "herede" las características de una ya existente. Por tanto, esta clase "heredera" o "hija" tendrá toda la funcionalidad de la clase de la que hereda (clase padre, siguiendo con la analogía), permitiendo además la inclusión de nuevos métodos y campos, así como la sobreescritura de los métodos ya existentes (si fuera necesario), para extender sus funcionalidades. De este modo, la clase resultante será una extensión de la clase que ya teníamos.

Por otro lado, cuando el sistema gestiona productos de la categoría ropa y electrónica, la clase `Usuario`, con la lista de productos que pone en venta un usuario, no necesitará saber si cada uno de los productos ofertados es de tipo *electrónica* o *ropa*. En este modo, las listas de objetos usamos un mecanismo llamado **Polimorfismo**, que permite que el sistema siga funcionando sin modificar el código que ya teníamos. Es decir, las llamadas a métodos de la clase `Usuario` que involucran productos, como pueden ser `addProducto()` o `setProductos()`, seguirán siendo válidas sin necesidad de modificar nada, y en este caso añadirán a la lista u obtendrán de ella el conjunto de productos ofertados por un usuario, independientemente de que la categoría del producto sea *ropa* o *electrónica*.

En esta etapa 3 estudiaremos el uso y posibilidades que nos ofrecen este tipo de mecanismos de la programación orientada a objetos.

4.1.2. Requisitos previos

Para la realización de esta etapa es necesario dominar los conceptos de objeto, clase y la relación entre clases denominada composición, estudiados hasta este punto.

4.1.3. Añadiendo productos de diferentes tipos

A diferencia de un producto de tipo *ropa*, que se caracteriza por tener una talla, los productos de la categoría *electrónica* se caracterizarán dentro de nuestro sistema por tener un peso y unas dimensiones expresadas en medidas de alto, largo y ancho. El conjunto de atributos que definían hasta ahora un producto (precio, descripción, estado y usuario) son compartidos por ambas categorías. En esta tercera etapa de la práctica se deben diseñar e implementar las clases necesarias para cubrir estas especificaciones.

Pero ahora hay que considerar un aspecto importante. Dado que ya no tendremos productos "en general", sino que serán de una categoría u otra, ya no tiene sentido que podamos instanciar objetos de la clase `Producto`, sino que habrá que hacerlo de una clase `ProductoElectronica` o `ProductoRopa`. Para ello, es necesario modificar la clase `Producto` que ya teníamos para convertirla en una **clase abstracta**, lo que significa que ya no se podrán instanciar objetos de la clase `Producto`. La finalidad de una clase abstracta es servir como clase base para otras clases que hereden de ella, ya que una clase abstracta recoge las características y funcionalidades comunes de todas sus clases hijas. Deberá considerarse con atención la visibilidad de los atributos de la clase `Producto` ahora que es una clase abstracta.

Como en las etapas previas, vamos a crear una nueva clase llamada `LanzadorEtapa3_1`, que contenga un método `main()`, cuyo cuerpo contenga el código fuente que se muestra a continuación[4]:

[4] En el curso virtual, junto con el enunciado de la práctica puede descargar el código fuente de esta clase LanzadorEtapa3_1

```
import java.util.List;

public class LanzadorEtapa3 {

    public static void main(String[] args) {

            // Creamos la plataforma de compraventa
            System.out.println("-----------------------------------");
            System.out.println("Creamos la plataforma de compraventa");
            System.out.println("-----------------------------------");
            PlataformaCompraVenta gestor = new PlataformaCompraVenta("Wallarock");

            // Creamos usuarios
            System.out.println("----------------");
            System.out.println("Creamos usuarios");
            System.out.println("----------------");
            Usuario user1 = new Usuario("Mariano Pérez Oso","marian0@hismail.com");
            Usuario user2 = new Usuario("Javier Carro Ñero","javicarronyero@othermail.com");
            // añadimos usuarios a la plataforma
            System.out.println("----------------");
            System.out.println("Añadimos usuarios");
            System.out.println("----------------");
            gestor.addUser(user1);
            gestor.addUser(user2);

            // Creamos productos
            System.out.println("----------------");
            System.out.println("Creamos productos");
            System.out.println("----------------");
            Producto producto1 = new ProductoElectronica(1,"Cámara digital Samsung",85,"Cámara de vídeo digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.",T_Estado.DISPONIBLE,6.5,11.01,2.15,0.6);
            Producto producto2 = new ProductoRopa(2,"Cazadora vaquera Levis Hombre",170.95,"Sin estrenar. Talla M",T_Estado.DISPONIBLE, "M");
            Producto producto3 = new ProductoRopa(3,"Cortavientos/chubasquero ADIDAS",250,"Color negro. Cremallera lateral de cuello a hombro. Talla S. Nuevo con etiqueta.",T_Estado.DISPONIBLE,"S");
            Producto producto4 = new ProductoElectronica(4,"Mini video cámara digital deportiva",20,"video cámara unotec gartner",T_Estado.DISPONIBLE,6.5,11.01,2.15,4.25);

            // Añadimos los productos a los usuarios
            System.out.println("-----------------------------------");
            System.out.println("Añadimos los productos a los usuarios");
            System.out.println("-----------------------------------");
            gestor.addProduct2User(producto1, "marian0@hismail.com");
```

```
gestor.addProduct2User(producto2, "javicarronyero@othermail.com");
gestor.addProduct2User(producto3, "javicarronyero@othermail.com");
gestor.addProduct2User(producto4, "marian0@hismail.com");


// Eliminamos un producto a un usuario
System.out.println("----------------------------------");
System.out.println("Eliminamos un producto a un usuario");
System.out.println("----------------------------------");
gestor.removeProduct2User(1,user1.getEmail());
gestor.printProductsByUser(user1.getEmail());

System.out.println("------------------------------");
System.out.println("Imprimimos todos los productos");
System.out.println("------------------------------");
gestor.printAllProducts();


// **************
// PARTE OPCIONAL
// **************

producto2.setEstado(T_Estado.RESERVADO);
// Imprimimos el conjunto total de productos que se encuentran en venta en la plataforma por estado
System.out.println("---------------------------------------");
System.out.println("Imprimimos los productos disponibles");
System.out.println("---------------------------------------");

gestor.printProductsByState(T_Estado.DISPONIBLE);


System.out.println("----------------------------------");
System.out.println("Imprimimos los productos reservados");
System.out.println("----------------------------------");
gestor.printProductsByState(T_Estado.RESERVADO);



// Imprimimos el conjunto de productos que ofrece un determinado usuario
System.out.println("-----------------------------------------------");
System.out.println("Imprimimos los productos disponibles de un usuario");
System.out.println("-----------------------------------------------");
gestor.printProductsByUser(user1.getEmail(),T_Estado.DISPONIBLE);
```

```
    }

}
```

Al ejecutar la clase anterior:

1. Se crea un objeto de la clase `PlataformaCompraVenta`.

```
-----------------------------------
Creamos la plataforma de compraventa
-----------------------------------
Creada plataforma de compraventa Wallarock
```

2. Se crean dos usuarios y cuatro productos correspondientes a las dos categorías que estamos considerando. A continuación, haciendo uso del polimorfismo, se incluyen los productos en la lista de productos en venta de los dos usuarios dos a dos, e independientemente de su categoría. Por otro lado, al incluir cada producto en las listas se fija por defecto su estado como *disponible*.

```
---------------
Creamos usuarios
---------------
Creado usuario marian0@hismail.com
Creado usuario javicarronyero@othermail.com
----------------
Añadimos usuarios
----------------
El usuario con email marian0@hismail.com ha sido añadido
El usuario con email javicarronyero@othermail.com ha sido añadido
----------------
Creamos productos
----------------
Creado producto con id 1
Creado producto con id 2
Creado producto con id 3
Creado producto con id 4
---------------------------------------
Añadimos los productos a los usuarios
---------------------------------------
El estado del producto con id 1 ha sido modificado a estado DISPONIBLE
El producto con id 1.0 se ha añadido al usuario marian0@hismail.com
El estado del producto con id 2 ha sido modificado a estado DISPONIBLE
El producto con id 2.0 se ha añadido al usuario javicarronyero@othermail.com
El estado del producto con id 3 ha sido modificado a estado DISPONIBLE
El producto con id 3.0 se ha añadido al usuario javicarronyero@othermail.com
El estado del producto con id 4 ha sido modificado a estado DISPONIBLE
El producto con id 4.0 se ha añadido al usuario marian0@hismail.com
```

3. Se elimina un producto de un usuario y se imprime el conjunto de productos que quedan en el sistema. Obsérvese cómo el mecanismo del polimorfismo nos permite acceder a los productos independientemente de la categoría:

```
-----------------------------------
Eliminamos un producto a un usuario
-----------------------------------
El producto con id 1.0 se ha eliminado de la lista del usuario marian0@hismail.com


ID: 4
Titulo: Mini video cámara digital deportiva
Precio: 20.0
Descripcion: video cámara unotec gartner
Estado: DISPONIBLE

Peso: 6.5 kg
Dimensiones (Alto x Largo x Ancho): 11.01 cm x 2.15 cm x 4.25 cm

------------------------------
Imprimimos todos los productos
------------------------------

ID: 4
Titulo: Mini video cámara digital deportiva
Precio: 20.0
Descripcion: video cámara unotec gartner
Estado: DISPONIBLE

Peso: 6.5 kg
Dimensiones (Alto x Largo x Ancho): 11.01 cm x 2.15 cm x 4.25 cm


ID: 2
Titulo: Cazadora vaquera Levis Hombre
Precio: 170.95
Descripcion: Sin estrenar. Talla M
Estado: RESERVADO

Talla: M


ID: 3
Titulo: Cortavientos/chubasquero ADIDAS
Precio: 250.0
Descripcion: Color negro. Cremallera lateral de cuello a hombro. Talla S. Nuevo con etiqueta.
```

Estado: DISPONIBLE

Talla: S

4. Se modifica el estado de un producto:

   El estado del producto con id 2 ha sido modificado a estado RESERVADO

5. Se imprimen por pantalla los productos en venta dentro del sistema y según su estado, obteniendo la siguiente salida:

   Imprimimos los productos disponibles

   ID: 1
   Titulo: Cámara digital Samsung
   Precio: 85.0
   Descripcion: Cámara de vídeo digital Samsung (wb800f) full HD 11,9 megapíxeles + wifi. Lente de 21x zoom , pantalla táctil con temporizador y muchas más funciones. Se incluye memoria micro SD, cargador Original, batería de litio y funda.
   Estado: DISPONIBLE

   Peso: 6.5 kg
   Dimensiones (Alto x Largo x Ancho): 11.01 cm x 2.15 cm x 0.6 cm

   ID: 4
   Titulo: Mini video cámara digital deportiva
   Precio: 20.0
   Descripcion: video cámara unotec gartner
   Estado: DISPONIBLE

   Peso: 6.5 kg
   Dimensiones (Alto x Largo x Ancho): 11.01 cm x 2.15 cm x 4.25 cm

   ID: 3
   Titulo: Cortavientos/chubasquero ADIDAS
   Precio: 250.0
   Descripcion: Color negro. Cremallera lateral de cuello a hombro. Talla S. Nuevo con etiqueta.
   Estado: DISPONIBLE

   Talla: S

Imprimimos los productos reservados

ID: 2

Titulo: Cazadora vaquera Levis Hombre

Precio: 170.95

Descripcion: Sin estrenar. Talla M

Estado: RESERVADO

Talla: M

6. Por último, se imprime el conjunto de productos en estado *disponible* que oferta para su venta un usuario:

Imprimimos los productos disponibles de un usuario

ID: 4

Titulo: Mini video cámara digital deportiva

Precio: 20.0

Descripcion: video cámara unotec gartner

Estado: DISPONIBLE

Peso: 6.5 kg

Dimensiones (Alto x Largo x Ancho): 11.01 cm x 2.15 cm x 4.25 cm

Al igual que en las etapas anteriores, es importante probar el código con diferentes datos y comprobar así el correcto funcionamiento del sistema.

FECHAS Y NORMAS DE ENTREGA

5.1. Sesiones presenciales

La realización de la práctica se llevará a cabo en los Centros Asociados, siendo las sesiones organizadas y supervisadas por el tutor de la asignatura. **Habrá como mínimo tres sesiones presenciales de obligatoria asistencia**. Los estudiantes contactarán con su centro asociado para informarse sobre cuándo tendrán que asistir a las sesiones.

A lo largo de las tres sesiones se abordarán las diferentes partes de las que consta la práctica. Las fechas orientativas para la realización de cada una de las etapas serán:

- Finales de marzo. Realización de la primera sesión presencial.
- Mediados de abril. Realización de la segunda sesión presencial.
- Principios de mayo. Realización de la tercera sesión presencial.

5.2. Normas de entrega

La práctica se entregará a través de una tarea definida en el entorno virtual de la asignatura. Cada estudiante creará y comprimirá una carpeta nombrada con su DNI y primer apellido separados por un guion (”8345385X-gonzalez”). Esta carpeta contendrá:

- Una memoria de no más de 6 páginas donde se expliquen las decisiones tomadas, así como el diseño (ver sección 1.2 de esta memoria) realizado en cada parte de la práctica.
- Los ficheros .java, sin caracteres especiales; por ejemplo “ñ” o tildes finales, es decir, únicamente los de la última etapa.

NOTA IMPORTANTE: Los nombres de los ficheros y carpetas/paquetes que compongan la práctica entregada deben contener SOLO caracteres correspondientes a las letras de la A a la Z, tanto mayúsculas como minúsculas, números del 0 al 9 y los caracteres especiales ’-’ y ’ ’. **No deben utilizarse otros, tales como tildes o símbolos**.

5.3. Fechas de entrega

Para la convocatoria ordinaria (junio) todos los estudiantes deberán entregar su práctica en el entorno virtual antes del 9 de mayo.

Los tutores deberán evaluar las prácticas y subir las notas al entorno virtual antes del 17 de mayo.

Las fechas correspondientes a la convocatoria extraordinaria (septiembre) se publicarán más adelante en el curso virtual.

NOTA IMPORTANTE: Los tutores deben cumplir unos requisitos ante los estudiantes porque la práctica cuenta para la calificación de la asignatura. Por tanto, antes de entregar las calificaciones definitivas al equipo docente deberán:

1. Informar de la nota de las prácticas a los estudiantes.

2. Dado que las prácticas forman parte de la evaluación del estudiante, establecer un día de revisión de notas de prácticas previo a la fecha tope de entrega de las calificaciones al equipo docente, establecido para el 17 de mayo,

Es importante que se mantengan todos los identificadores definidos en el enunciado, es decir, el nombre de las clases, atributos y métodos deben ser tal y como se definen en este enunciado.

5.3. Evaluación de la práctica

Las prácticas tienen carácter **INDIVIDUAL**, para evitar posibles copias todas las prácticas pasarán por un software detector de ejemplares similares. La detección de prácticas copiadas implicará un **SUSPENSO en TODO el curso, es decir, convocatorias de junio y septiembre, para todos los implicados.**
Es requisito indispensable para aprobar el examen la asistencia a las 3 sesiones obligatorias de la práctica, así como superar la propia práctica.
Las prácticas NO se guardan de un curso para otro.

5.4. Preguntas al equipo docente

Las preguntas sobre la instalación del entorno de desarrollo, puesta en funcionamiento y errores de compilación deben remitirse a los tutores de los centros asociados. El equipo docente atenderá preguntas de carácter metodológico y de diseño, preferentemente a través de los foros del curso virtual de la asignatura y del siguiente correo electrónico:

fund_inf@lsi.uned.es

Los datos de contacto de cada uno de los miembros del equipo docente son:

Roberto Centeno Sánchez (coordinador de la asignatura), *Profesor Titular de Universidad*
Tutorías: Jueves de 11:00 a 13:00h. y de 15:00 a 17:00 h.
Teléfono: 91 398 96 96
Mail: rcenteno@lsi.uned.es

Víctor Fresno Fernández, *Profesor Titular de Universidad*
Tutorías: Martes y Miércoles de 11:30 a 13:30
Teléfono: 91 398 82 17
Mail: vfresno@lsi.uned.es

Enrique Amigó Cabrera, *Profesor Titular de Universidad*
Tutorías: Jueves de 15:00 a 19:00
Teléfono: 91 398 86 51
Mail: enrique@lsi.uned.es
enrique@lsi.uned.es

Alberto Pérez García-Plaza, *Profesor Permanente Laboral*
Tutorías: Jueves de 11:00 a 15:00h
Teléfono: 91 398 84 12
Mail: alberto.perez@lsi.uned.es